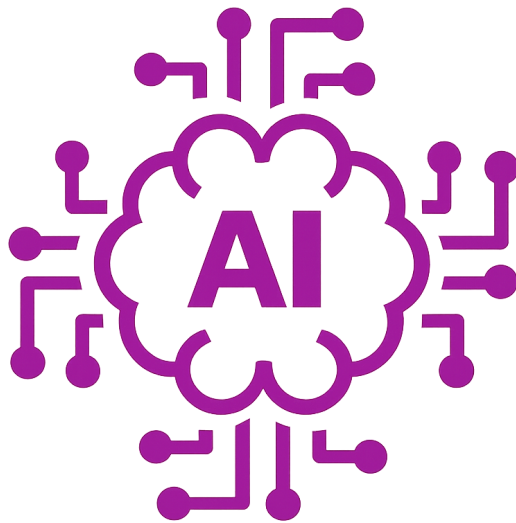


ML / DL : PBL 04

« Построение модели для поиска вредоносных
обращений по журналу веб-сервера »



SK Shieldus

Rookies, 30-й набор (личный номер 14)

Ким Джэ-Ук

Решение PBL 04

```
import re
from datetime import datetime
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, accuracy_score
from sklearn.metrics import precision_score, recall_score, f1_score
```

```
# загружаем данные и смотрим, как они устроены
log_df = pd.read_csv("web_server_logs_2.csv")

# бегло осматриваем таблицу
print(f"В этой таблице число строк {log_df.shape[0]} и "
      f"число столбцов {log_df.shape[1]}")
print("-"*30)
print("Выведем все столбцы, какие тут есть\n"
      f"{'', '.join(log_df.columns.values.astype(str)[:])}")
print("-"*30)
# посмотрим на первые четыре строки
log_df.head(4)
# в Quarto таблица DataFrame выводится через средства Jupyter,
# поэтому она получается опрятной и без отдельной настройки HTML
```

В этой таблице число строк 100 и число столбцов 3

Выведем все столбцы, какие тут есть
timestamp, status_code, size

	timestamp	status_code	size
0	2026-01-03T09:00:00+09:00	200	1875
1	2026-01-03T09:00:05+09:00	404	998
2	2026-01-03T09:00:10+09:00	404	1627
3	2026-01-03T09:00:15+09:00	403	1139

```
# посмотрим, есть ли пропуски в каждом столбце
print(f"{'(s := log_df.isna().sum()).to_dict()}' ==> "
      f"всего пропусков в этих данных {s.sum()}")
```

{'timestamp': 0, 'status_code': 0, 'size': 0} ==> всего пропусков в этих данных 0

※ Запись +09:00 в конце **timestamp** означает, что это время опережает всемирное на девять часов, то есть это корейское время.

```
from IPython.display import HTML

# делаем таблицу из переменных, то есть столбцов, и их типов
dtype_map = log_df.dtypes.astype(str)
html_table = pd.DataFrame({"Feature": log_df.columns,
                           "Тип": [dtype_map[c] for c in log_df.columns]})

# добавляем столбец Index, нумерация идёт с единицы
html_table.insert(0, "Index", range(1, len(html_table) + 1))

# выводим таблицу в виде HTML
HTML(html_table.to_html(index=False, border=1))
```

Index	Feature	Тип
1	timestamp	object
2	status_code	int64
3	size	int64

**Какие переменные (Feature) разумно перевести в набор нулей и единиц ? -
прежде всего о том, как иначе определить вредоносное обращение -**

Прежде всего вспомним, что перевод в набор нулей и единиц, то есть однократное кодирование, это способ подготовки данных, при котором для каждого значения качественной переменной заводится свой столбец, и там ставится 1, если наблюдение относится к этому значению, и 0, если нет.

Далее, как же тогда в теме нашего задания « **построение модели для поиска вредоносных обращений по журналу веб-сервера** » можно определить ‘**вредоносное обращение**’ на основе того, что есть в данных?

Учитывая, что данные ограничены, мы определили ‘**вредоносное обращение**’ иначе, а именно как **образ поведения, при котором раз за разом появляются необычные ошибочные ответы HTTP в течение короткого промежутка времени**. Если говорить точнее, в этом наборе данных нет ключевых для безопасности сведений, таких как адрес обращения URI, адрес клиента IP, способ обращения HTTP или User-Agent. Поэтому строго определить вредоносность отдельного обращения, опираясь только на три переменные набора, а это **timestamp** (= время, когда обращение дошло до сервера, то есть когда именно возникла эта запись журнала), **status_code** (= код состояния HTTP, который сервер вернул на это обращение, он показывает, прошло ли обращение, было ли отклонено или случилась ошибка) и **size** (= объём ответа, отправленного клиенту на это обращение), на деле почти невозможно. Поэтому в этом задании мы решили понимать ‘**вредоносное обращение**’ не как отдельное событие, а как итог совместного поведения, у которого есть протяжённость во времени.

Эта 'протяжённость во времени' и означает в определении '**вредоносного обращения**' **короткий промежуток времени**, а именно мы взяли **последние 30 секунд**. А 'совместное поведение' означает в этом определении **образ поведения, при котором раз за разом появляются необычные ошибочные ответы HTTP**, и здесь мы взяли **случай, когда за последние 30 секунд от данной точки набирается три и более кода состояния 4xx**. Такие случаи мы помечаем как вредоносные, исходя из того, что это ближе не к поведению обычного пользователя, а к **самодействующим необычным обращениям** вроде **поиска ресурсов, перебора путей, повторных попыток доступа или попыток обойти проверку доступа**. Всё это тесно связано с простейшим видом поиска угроз по поведению, который применяют на деле, а как заметный пример самодействующих необычных попыток мы держим в уме **нападение перебором**.

Нападение перебором (Brute force)

Нападение, при котором самодействующим образом одно за другим подставляются все возможные пароли или проверочные значения, и так до тех пор, пока не найдётся подходящее

```
# смотрим неповторяющиеся значения status_code, то есть быстро
# проверяем, какие коды состояния тут вообще есть
print("Неповторяющиеся значения в столбце status_code ",
      ", ".join(map(str,
                    sorted(log_df["status_code"].dropna().unique().tolist()))))
```

Неповторяющиеся значения в столбце status_code 200, 302, 403, 404

Значение status_code

- 200 (OK) значит, что обращение прошло как надо и сервер без затруднений вернул то, что просили
- 302 Found (Redirect) значит, что запрошенное на время переехало в другое место. Обозреватель сам переходит по новому адресу, который указал сервер
- **403 Forbidden** значит, что сервер понял обращение, но отказал, ведь права доступа отсутствуют. Так бывает, когда нет проверки доступа, когда прав не хватает или когда обращение закрыто правилами безопасности
- **404 Not Found** значит, что запрошенного на сервере (страницы, файла, пути) не существует. Так бывает при неверном URL или при обращении к удалённой странице

```

# создаём новую переменную, а именно достаём час обращения (hour)
# и добавляем его отдельным столбцом
log_df["timestamp"] = pd.to_datetime(log_df["timestamp"], errors="coerce")

# убеждаемся, что timestamp записан строго по порядку времени,
# и это важно для дальнейшей работы
bad = log_df["timestamp"].diff().dt.total_seconds() < 0
print(f"Сколько раз время идёт вспять {int(bad.sum())}")
print(f"***777")
print("Данные timestamp расставлены по порядку времени"
      if int(bad.sum()) == 0 else "Данные не расставлены по порядку времени")

```

Сколько раз время идёт вспять 0

Данные timestamp расставлены по порядку времени

Проверить, расставлен ли timestamp по порядку появления записей, очень важно.

Дело в том, что через *diff()* < 0 в коде мы находим места, где время возвращается назад по сравнению с предыдущей строкой, и тем самым **ручаемся, что применяемые дальше действия в окне по времени, такие как *rolling("30s")*, верно отражают настоящий ход времени**. Если этот порядок нарушен, само понятие «последние 30 секунд» перестаёт существовать, число накопленных ошибок и создание метки искажаются, а вместе с этим рушится и надёжность обучения модели.

```

# достаём час из timestamp, чтобы затем перевести время суток
# в набор нулей и единиц и подать как переменную
log_df["hour"] = log_df["timestamp"].dt.hour

# вредоносное обращение ( label ) это "три и более 4xx
# за последние 30 секунд"
# ставим признак ошибки по тому, больше ли код состояния HTTP,
# чем 400
log_df["is_error"] = (log_df["status_code"] >= 400).astype(int)

# считаем число накопленных ошибок за 30 секунд, то есть в окне
# по времени
rolling_err_30s = (log_df.set_index("timestamp")["is_error"]
                    .rolling("30s").sum().reset_index(drop=True))

# присоединяем это к таблице как столбец
log_df["err_cnt_30s"] = rolling_err_30s.values

# создаём метку вредоносного обращения, а именно при трёх и более
# ошибках за последние 30 секунд это вредоносное ( = 1 ),
# иначе обычное ( = 0 )
log_df["label"] = (log_df["err_cnt_30s"] >= 3).astype(int)

# берём от size логарифм, причём через log1p, ведь там может быть ноль
log_df["size_log"] = np.log1p(pd.to_numeric(log_df["size"], errors="coerce"))

```

Перевод в набор нулей и единиц (**однократное кодирование) выполнен с оглядкой на **утечку ответа (Target leakage)**.**

Переменные *is_error* , *status_code* и *err_cnt_30s* прямо участвуют в создании метки, и если включить их в число входных переменных, может возникнуть **утечка ответа (Target leakage)**. Поэтому вместо них мы взяли *hour* и *size_log*, разбили их на промежутки в согласии со смыслом задания (*hour_bin* и *size_bin*) и тем самым сделали качественными, а затем перевели их в набор нулей и единиц и использовали как входные переменные, чем и выполнили требование о кодировании и о приведении к одному масштабу.

```

## переводим в набор нулей и единиц

# hour -> hour_bin, то есть разбиваем время суток
# на осмысленные промежутки
# 0 - 5 ночь, 6 - 11 утро, 12 - 17 день, 18 - 23 вечер
log_df["hour_bin"] = pd.cut(log_df["hour"], bins=[-1, 5, 11, 17, 23],
                             labels=["night", "morning", "afternoon", "evening"])

# size_log -> size_bin, то есть делим на три промежутка

```

```
# по распределению самих данных, то есть по квантилям
log_df["size_bin"] = pd.qcut(log_df["size_log"],q=3,
                             labels=["small", "medium", "large"])

# перевод в набор нулей и единиц, то есть качественная переменная
# становится столбцами из 0 и 1
hour_bin_ohe = pd.get_dummies(log_df["hour_bin"], prefix="hour", dtype=int)
size_bin_ohe = pd.get_dummies(log_df["size_bin"], prefix="size", dtype=int)

# готовим данные для обучения и отбираем главные переменные,
# где hour это время суток обращения,
# а size_log это логарифм объёма ответа
feature_df = pd.concat([log_df[["hour", "size_log"]], hour_bin_ohe,
                        size_bin_ohe], axis=1)
```

```
# делаем таблицу про переменные, то есть про столбцы
dtype_map = feature_df.dtypes.astype(str)
html_table = pd.DataFrame({"Feature": feature_df.columns,
                           "Тип": [dtype_map[c] for c in feature_df.columns]})

# добавляем столбец Index, нумерация идёт с единицы
html_table.insert(0, "Index", range(1, len(html_table) + 1))

# выводим таблицу в виде HTML
HTML(html_table.to_html(index=False, border=1))
```

Index	Feature	Тип
1	hour	int32
2	size_log	float64
3	hour_night	int64
4	hour_morning	int64
5	hour_afternoon	int64
6	hour_evening	int64
7	size_small	int64
8	size_medium	int64
9	size_large	int64

Переменные в таблице выше созданы для того, чтобы подать модели сведения о времени и об объёме ответа из исходного журнала. Сначала **hour** это числовая переменная, куда из timestamp взят только час обращения (от 0 до 23), а **size_log** это непрерывная переменная, где size переведён в число, а затем взят его логарифм через `log1p`, чтобы можно было работать и с нулём. Далее **hour_night**, **hour_morning**, **hour_afternoon** и **hour_evening** это переменные из нулей и единиц, полученные так, что hour разбит на четыре промежутка времени суток, а именно 0 – 5 (ночь), 6 – 11 (утро), 12 – 17 (день), 18 – 23 (вечер), и затем эти промежутки

переведены в набор нулей и единиц , а **size_small** , **size_medium** и **size_large** это переменные из нулей и единиц, полученные так, что распределение **size_log** поделено по квантилям на три равные части (малый, средний, большой), и эти промежутки тоже переведены в набор нулей и единиц.

```
# выбираем метку
label_series = log_df["label"]

# делим данные на обучающую и проверочную части ( 70% и 30% )
X_train, X_test, y_train, y_test = train_test_split(
    feature_df,
    label_series,
    test_size=0.3,
    random_state=42,
    stratify=label_series
)

# обучаем модель Logistic Regression
clf = LogisticRegression(max_iter=1000)
clf.fit(X_train, y_train)

# получаем предсказания на проверочных данных
y_pred = clf.predict(X_test)

# считаем и выводим оценки качества
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, zero_division=0)
recall = recall_score(y_test, y_pred, zero_division=0)
f1 = f1_score(y_test, y_pred, zero_division=0)

print(f'Доля верных ответов ( Accuracy ) {accuracy:.4f} ')
print(" "*30)
print(f'Точность ( Precision ) {precision:.4f} ')
print(" "*30)
print(f'Полнота ( Recall ) {recall:.4f} ')
print(" "*30)
print(f'F1 - Score {f1:.4f}')
```

Доля верных ответов (Accuracy) 0.5667

Точность (Precision) 0.5862

Полнота (Recall) 0.9444

F1 - Score 0.7234

Толкование результатов и их значение

Если учитывать смысл этого задания, а именно « на основе времени суток (**hour**) и объёма ответа (**size**) из журнала веб-сервера подготовить данные и применить логистическую регрессию (**Logistic Regression**) для разделения вредоносных обращений (**label**), причём **подобаяюще и в условиях ограниченного набора данных**, а затем проверить **качество** », то рассмотреть приведённые ниже более подробные **толкование результатов и их значение** тоже будет осмысленно.

- (1) Прежде всего, из timestamp в данном файле csv было получено **время суток обращения (hour)**, а случаи, когда за последние 30 секунд набирается три и более кода состояния **4xx**, были помечены как **вредоносное обращение (label = 1)**. Для этого по тому, больше ли **status_code** чем 400, был создан **признак ошибки (is_error)**, затем в окне по времени длиной 30 секунд, отсчитываемом по timestamp, было посчитано **число накопленных ошибок (err_cnt_30s)**, и метка была создана по условию $err_cnt_30s \geq 3$. Такой способ разметки можно считать попыткой уловить **вредоносное обращение** через **необычное поведение (Behavior-based anomalous activity pattern)**, а именно через **повторное появление неудачных ответов (4xx) в течение короткого промежутка времени**, в таких условиях, как в нашем задании, где по одному обращению судить о нападении нельзя из-за ограниченности данных. Кроме того, это можно толковать как замысел, при котором, пусть и ограниченно, держится в уме **нападение перебором**, то есть **способ нападения, при котором проверочные значения (пароли и подобное) подставляются самодействующим образом много раз подряд, из-за чего возникает множество неудачных ответов**. При этом с оглядкой на утечку ответа (Target leakage) переменные **is_error**, **status_code** и **err_cnt_30s** были исключены из входных, а на основе **hour** и переведённого в число **size**, то есть **size_log**, были созданы переменная с промежутками времени суток (**hour_bin**) и переменная с промежутками объёма ответа (**size_bin**), которые затем были **переведены в набор нулей и единиц** и поданы модели как входные переменные. Затем данные были поделены на обучающую и проверочную части (70 % и 30 %). Значение такого устройства опыта в том, что **в условиях ограниченного набора данных он позволяет получить осмысленный итог, сохраняя при этом подобающий порядок работы в машинном обучении и тщательно предупреждая утечку ответа (Target leakage)**.
- (2) Показатели качества этой модели (Accuracy, Precision, **Recall**, F1-score) численно показывают, **насколько предсказания совпадают с настоящими метками на проверочных данных**. А именно, Accuracy это доля верных ответов среди всех предсказаний, Precision это доля по-настоящему вредоносных среди тех, что модель сочла вредоносными ($= 1$), Recall это доля тех по-настоящему вредоносных ($= 1$), которые модель нашла, а F1-score показывает равновесие между Precision и Recall. Если посмотреть на полученные значения, Accuracy равна **0.5667**, то есть в целом **доля верных ответов не такая уж высокая**, а Precision равна **0.5862**, и это указывает, что среди предсказанных вредоносными ($= 1$) немалая часть может оказаться обычными ($= 0$). Напротив, **Recall** равна **0.9444**, то есть очень высока, а значит, предсказания смещены в сторону того, чтобы не упустить настоящие вредоносные обращения ($= 1$)

и поймать почти все. То, что **Recall** выходит заметно выше остальных показателей, объяснялось в части про машинное обучение в ходе нашего обучения на примере того, как задача разделения применяется к распознаванию рака. В частности, отмечалось, что высокая **Recall** полезна там, где важно не упустить больного. Схожим образом и в этом опыте, с той точки зрения, что **важно не упустить вредоносное обращение**, пусть работа и велась в довольно **ограниченных условиях по данным**, **появляется возможность толковать вид этих показателей благоприятно.**

Разбор ограничений и предложения по улучшению

Если учитывать смысл этого задания, а именно « на основе времени суток (**hour**) и объёма ответа (**size**) из журнала веб-сервера подготовить данные и применить логистическую регрессию (**Logistic Regression**) для разделения вредоносных обращений (**label**), причём **подобаяюще и в условиях ограниченного набора данных**, а затем проверить качество » , то приведённые ниже более подробные **разбор ограничений и предложения по улучшению** тоже оказываются осмысленными.

- (1) Такое сочетание, при котором **Recall** заметно выше прочих показателей, приводит к толкованию, что ложных срабатываний (обычное сочтено вредоносным) скорее всего было довольно много. В итоге F1-score равна **0.7234**, и это одной величиной описывает склад поиска так, что «вредоносных обращений находится много (впереди идёт **Recall**), но точностью (**Precision**) при этом пожертвовали». Поэтому точнее будет толковать этот итог так, что **при нынешнем определении метки (три и более 4xx за последние 30 секунд)** сочетание переменных, построенных на времени суток (**hour**) и объёме ответа (**size**), работало в сторону наименьшего числа пропусков вредоносного, но **в части уменьшения ложных срабатываний оказалось ограниченным**. Такое **ограничение** по сути **вытекает из того, что данный файл csv устроен просто и состоит лишь из *timestamp* , *status_code* и *size*.**
- (2) Кроме того, **метка (**label**)** в этом опыте определена правилом подсчёта по времени, а именно **через число накопленных ошибок за последние 30 секунд**, и есть **ограничение в том, что входными переменными служат только промежутки **hour** и **size_log**, переведённые в набор нулей и единиц**. То есть у модели есть основное **ограничение**, идущее от ограниченности набора данных, ведь она **не может учесть** другие ключевые для безопасности входные переменные, без которых поиск **вредоносных обращений** обойтись не может (адрес обращения URI, адрес клиента IP, способ обращения HTTP, User - Agent и прочее). Иначе говоря, эта модель угадывает вредоносный образ поведения лишь по времени суток и по распределению объёма ответа. Поэтому даже при высоких показателях правильнее толковать это не как «нападение опознано напрямую», а как вывод о том, что **время суток обращения (**hour**) и объём ответа (**size**) статистически связаны с вредоносными обращениями**. И всё же есть смысл в том, что **в условиях ограниченного набора данных** меткой удалось довольно осмысленно приблизить тот образ поведения, которого ждёшь от самодействующих повторных попыток вроде **перебора**, а именно **накопление неудачных ответов за короткое время**. То есть значение этой работы в том, что **способ мышления при поиске угроз по поведению в стеснённых условиях был подобающе** применён к модели машинного обучения. Наконец, можно подытожить, что **для того, чтобы ещё сильнее усилить переносимость и качество модели в согласии со смыслом задания, стоит дополнительно добыть ключевые для безопасности сведения, такие как адрес обращения URI, адрес клиента IP, способ обращения HTTP и User - Agent, и в дальнейших разборах и работах положить в**

основу разметки и построения переменных отдельные признаки вредоносности (например, повторные неудачи проверки доступа, шквал обращений с одного IP, перебор определённых путей, образ доступа). По этому вопросу мы опирались на источник [1] *Web Server Logs Preprocessing for Web Intrusion Detection*.

Главное, что пришлось обдумать, это предупреждение утечки ответа (Target leakage) и определение вредоносного обращения

Держа в уме точное название задания « построение модели для поиска вредоносных обращений по журналу веб-сервера », при работе над ним особенно внимательно пришлось отнестись к утечке ответа (Target leakage) и к определению вредоносного обращения.

Первое, что пришлось обдумать, это предупреждение утечки ответа (Target leakage), а именно стоит ли переводить в набор нулей и единиц переменную *status_code*

Учитывая тему задания « построение модели для поиска вредоносных обращений по журналу веб-сервера » и замысел составителя, поначалу мы предварительно решили, что не переменную *timestamp*, а переменную *status_code* стоит превратить в качественную (*status_class*), создав её заново, и уже её перевести в набор нулей и единиц. Так вышло потому, что *timestamp* по своему смыслу это, по сути, непрерывные сведения о времени, а значит, черты качественной переменной у него слабые. И напротив, у *status_code* по самому стандарту HTTP изначально есть строение смысла, которое хорошо ложится на качественную переменную.

Поэтому ради перевода в набор нулей и единиц переменная *status_code* была превращена в *status_class_2* или *status_class_3* или *status_class_4*. Так сделано потому, что по стандартному смыслу кодов состояния HTTP коды, начинающиеся с 2, 3, 4 и 5 (кодов из пятой сотни в этом наборе данных нет, но опишем их ради знания области), несут разный смысл. Если говорить точнее, коды, начинающиеся с 2, такие как 200, 201, 204, означают, что обращение прошло как надо, поэтому 2xx можно собрать в разряд «успех». Далее, коды, начинающиеся с 3, такие как 301, 302, 304, означают, что запрошенное переехало в другое место или что следует взять сохранённое, и там часто пользователя отправляют на другой URL, поэтому 3xx можно отнести к разряду «перенаправление». Коды, начинающиеся с 4, такие как 400, 403, 404, можно отнести к разряду «ошибка клиента», ведь либо с самим обращением что-то не так, либо нет прав (403), либо запрошено то, чего не существует (404). Наконец, коды, начинающиеся с 5, такие как 500, 502, 503, можно отнести к разряду «ошибка сервера», ведь это внутренние затруднения сервера, недоступность шлюза или службы, то есть неудача обработки на стороне сервера.

По этим на вид (?) убедительным причинам переменная *status_code* была превращена в качественную (*status_class*), создана заново и переведена в набор нулей и единиц, и в итоге **Accuracy, Precision, Recall и F1 все вышли равными 1, то есть получился неудачный итог, при котором есть подозрение на утечку ответа (Target Leakage)**. Так вышло потому, что превращение *status_code* в качественную переменную (*status_class*) и перевод её в набор нулей и единиц в итоге дали модели в качестве входной переменной (feature) сведения, которые прямо или почти прямо намекают на ответ (label). То есть с точки зрения модели складывается такое устройство, при котором сложный образ учить не нужно, ведь достаточно взглянуть на *status_class*, чтобы почти безошибочно угадать **label** ...

Второе, что пришлось обдумать, это как подобающе определить **вредоносное обращение** при таких ограниченных данных

Учитывая тему задания « построение модели для поиска вредоносных обращений по журналу веб-сервера » и замысел составителя, сводить определение **вредоносного обращения** просто к появлению кодов состояния HTTP из четвёртой сотни (а именно, если опираться на данный набор данных, к 403 и 404 , подробнее о каждом из них уже сказано вверху страницы 3) неуместно. Так происходит потому, что такие ошибки нередко возникают и при обычных намерениях, в обычных условиях работы, и это дело естественное. Поэтому, внимательно вглядевшись в смысл задания, мы определили вредоносное обращение не просто через коды состояния HTTP из четвёртой сотни, а как **образ поведения, при котором раз за разом появляются необычные ошибочные ответы HTTP в течение короткого промежутка времени**, и на этом определении построили весь опыт. Прежде всего мы сочли, что в этих **ограниченных условиях по данным** это самый действенный способ распорядиться переменной timestamp, и уверились, что именно ради этого она и была дана. Если память нас не подводит, в ходе нашего обучения мельком заходила речь о разделении **вредоносных обращений** по частым обращениям в определённое время суток, и это тоже во многом помогло при работе над заданием.

Этот документ подготовлен на основе формата **Quarto Markdown** (**.qmd**), который позволяет объединять код **Python** и текст документа в одном файле, и выведен в формате PDF.

Использованные материалы

- [1] S. E. Salama, M. I. Marie, L. M. El-Fangary, and Y. K. Helmy, "Web server logs pre-processing for web intrusion detection," *Computer and Information Science*, 2011, doi: [10.5539/cis.v4n4p123](https://doi.org/10.5539/cis.v4n4p123).